OPERATOR GUIDE · LAPTOP TO LIVE

Start a CPU Miner on Your Own Machine

The complete workflow, in order: connect your laptop, create and fund the wallet, run the setup from the Deploy stepper, pass the free Validator Lab gate — and only then burn to register. Seven steps, one burn, $0 in rent.

Target subnet — SN67 · Harnyx (CPU-only, agents & logic)
Platform — Infranex control plane · Deployments · Validator Lab · DevOps
Prepared — September 20, 2026

01 · THE MAP

The workflow at a glance

Mining SN67 Harnyx with your laptop is a **sequence**, not a pile of screens. Three things must exist before any money moves: your machine must be reachable by the app, your wallet keys must exist **on that machine**, and the subnet's software must be installed there. Only after a free pre-flight passes do you spend anything — one burn of roughly **0.0433 TAO (≈ $11.62 at $268.33/TAO, block 9,101,516)** to register. Everything else in the workflow is free.

The order below is deliberate. The laptop comes first because every later command — wallet creation, subnet registration, the miner itself — executes **on your machine** through the agent you enroll in step 1. The wallet comes before registration because the register command consumes TAO from the coldkey that lives on the laptop. The Validator Lab gate comes before the burn because registration is irreversible: once a hotkey takes a seat on SN67, that TAO is gone whether or not your agent ever earns. Follow the order and you never pay before you are ready.

1 **Connect your local machine** FREE

CPU Catalog → "Or skip the cloud" → name it, run two commands in Ubuntu (WSL2 on Windows). The agent dials out to the app — no open ports, no inbound firewall rules.

2 **Create & fund the wallet** FREE NEEDS TAO

Generate coldkey + hotkey on the laptop with `btcli`, back up the mnemonic on paper, then transfer ~0.1 TAO to the coldkey so the burn in step 5 has funds.

3 **Pick SN67 & run setup** FREE

Deployments stepper: step 1 pick SN67, step 2 "Local machine", step 3 "Start setup on laptop" — the agent clones the Harnyx repo and installs the Python 3.11 stack.

4 **Pass the Validator Lab gate** FREE

Sync netuid 67, read the validator's scoring shape, simulate your agent, iterate until the verdict is competitive. **No green verdict — no burn.**

5 **Register — the ONE burn** BURN ≈ $11.62

Step 4 of the stepper: "Send register command to laptop". This takes a seat on SN67 and is irreversible — the gate from step 4 must be green first.

6 **Submit the agent & go live** FREE

Package your agent through the harnyx-miner-sdk per the repo's own flow. Validators execute it in sandboxes and score it every epoch.

7 **Monitor & earn** FREE

Weekly check: Opportunities strip, Harnyx dashboard for your agent's rank, taostats for real payouts. If the deal dies, deregister and move the burn elsewhere.

01 · CONTINUED

When to do what — the master schedule

This table answers the three questions every first-time operator asks: **when do I connect the wallet, when do I connect the local machine, and when do I register?** Print it or keep it open — each row links back to the step where the full instructions live.

Milestone	Where in the app	When	Cost
Connect local machine	CPU Catalog → "Or skip the cloud"	First — once, before anything else	`free`
Create cold + hot keys	Terminal on the laptop (btcli)	After the machine is online	`free`
Fund the coldkey	Exchange or your main TAO wallet	Right after key creation, before step 5	`~0.1 TAO` moved
Wallet / hotkey fields	Deploy stepper (prefilled)	Step 3 review, before launching	`free`
Validator Lab gate	Validator Lab → netuid 67	Before ANY burn — no exceptions	`free`
Register on-chain	Deploy stepper → step 4 burn gate	Only after the verdict is green	`burn 0.0433 τ`
Submit the agent	harnyx-miner-sdk CLI on laptop	Immediately after registration	`free`
Monitor & review	Opportunities, Monitoring, DevOps	Weekly, forever	`free`

> **Why the wallet must be created on the laptop:** the register command runs through your machine's agent, so `btcli` looks for the wallet files in `~/.bittensor/wallets/` **there**. Keys created on another computer are invisible to it. Never paste a mnemonic into the app — it stores metadata only.

02 · PREPARATION

Before you start — the checklist

The whole workflow takes 30–60 minutes the first time, spread across your browser and one terminal window. Nothing in it needs a GPU, Docker, or CUDA — SN67 is an agents-and-logic subnet where **validators run your script in their own sandboxes**, so your laptop only has to be good enough to hold the repo and talk to the network. Confirm every item below before step 1; each one exists because skipping it is the usual cause of a failed or stalled deployment later.

- ☐ **A laptop or desktop with 2+ CPU cores, 4–8 GB RAM and ~10 GB free disk.** The Harnyx stack is a Python workspace, not a model — typical CPU-subnet requirements are tiny compared to a GPU box (the catalog's floor is 2 vCPU / 4 GB / 40 GB, and SN67 needs well under that).

- [ ] **Ubuntu available.** On Windows: open PowerShell and run `wsl --install -d Ubuntu`, then reopen and continue in Ubuntu. On Linux or macOS the native terminal works as-is.
- [ ] **A stable internet connection.** The agent pulls commands over HTTPS and the miner talks to the network continuously — a machine that sleeps or disconnects stops earning.
- [ ] **Access to the Infranex app** — sign in with your user and access code. Every screen named in this guide (CPU Catalog, Deployments, Validator Lab, DevOps) lives in the sidebar.
- [ ] **~0.1 TAO reachable** on an exchange or an existing wallet you control. This is **transferred**, not spent: ~0.0433 TAO covers the registration burn and the rest stays in your coldkey as buffer and fees.
- [ ] **Honest expectations.** SN67 recently showed ~117–222 rewarded seats, a burn of 0.0433 τ and per-earning-miner estimates that ranged widely across reads (~$100/mo net base-case to ~$1,282/mo in fresher snapshots). Emissions move — re-read the live Opportunities row before committing, and treat every number in this guide as a snapshot of September 2026.

> **Why SN67 is the recommended target:** it is the best CPU-classified subnet in the fresh scan — no GPU needed, no Docker, no CUDA, validators execute your agent, ~117 miners currently rewarded, and the cheapest meaningful entry burn in the network (≈ $11.62 vs ~1 τ on heavier subnets). Runner-ups: SN45 AlphaRidge and SN62 Ridges.

03 · STEP 1 — DAY ZERO

Connect your local machine (your laptop)

Do this **first**. Every command in the rest of this guide — setup, wallet creation, the register burn — is queued through the Infranex agent that you install in this step. The agent is a small Python process that makes **outbound-only** HTTPS calls to the app, signs every request with HMAC, and pulls its own command queue. You never open a port on your router and never expose SSH; if you close the terminal, the machine simply shows offline until you start it again.

IN APP **CPU Catalog** → scroll to "Or skip the cloud — connect your local machine" (the same flow also lives in **DevOps → Local machines**)

Enter a machine name (something you will recognize later, like `hp-ultra5-laptop`) and press **Connect my laptop**. The app mints a one-time

enrollment token and shows two commands. Run them in your Ubuntu terminal — paste the first exactly as shown (the token is valid for 30 minutes and works once), then start the agent with the second:

```
# 1 — enroll (the app shows this filled with your real server + token)
curl -fsSL <app-url>/api/agent/agent.py -o /tmp/infranex-agent.py && \
  python3 /tmp/infranex-agent.py enroll --server <app-url> --token <ONE-TIME-TOKE
N>

# 2 — start the agent (keep it alive)
python3 /tmp/infranex-agent.py run

# keep-alive option: run inside tmux, or:
nohup python3 /tmp/infranex-agent.py run &
```

Within a few seconds the machine flips to an **online** chip in the catalog and in DevOps → Local machines, together with its specs (CPU model, cores, RAM, OS, Python version). The agent polls the app roughly every ten seconds, so a queued command lands on your laptop almost immediately and its output streams back into the app. Revoke the machine at any time from DevOps to cut its access instantly.

> **Keep it running:** the agent must stay alive for the machine to stay online. Use `tmux` or the `nohup … &` form above, and disable automatic sleep on the laptop — a suspended machine cannot mine and cannot register. Windows users: run everything inside the WSL2 Ubuntu window and leave it open.

04 · STEP 2 — KEYS & FUNDS

Create and fund the wallet (before any register)

A Bittensor wallet is two keys: a **coldkey** that holds your funds and signs, and a **hotkey** that does the work and serves. Both are generated locally with `btcli` — on the laptop, not in the app and not on another computer — because the register command in step 5 runs there and looks for the key files in `~/.bittensor/wallets/`. The names you pick here are exactly the ones you will see prefilled in the Deploy stepper later; this guide uses `harnyx` and `sn67miner`. If `btcli` is missing, install it first with `pip install bittensor` (the Harnyx venv from step 3 also ships it).

```
# run these on the laptop, inside Ubuntu
btcli wallet new-coldkey --wallet.name harnyx --wallet.path ~/.bittensor/wallets
btcli wallet new-hotkey  --wallet.name harnyx --wallet.hotkey sn67miner

# check the addresses when done
btcli wallet overview --wallet.name harnyx
```

When the coldkey is generated, `btcli` shows a mnemonic — a sequence of words that **is** the wallet. Write it on paper and store it offline; if it leaks, your TAO leaks, and no support desk can restore it. Copy the two ss58 addresses into the app's **Wallets** view for reference — it stores records and metadata only, never mnemonics.

Fund the coldkey now, while you are here: withdraw or transfer about **0.1 TAO** from your exchange or main wallet to the coldkey's ss58 address. The transfer itself is not a cost — it becomes the burn in step 5 (~0.0433 τ) plus a safety margin for fees and price movement. A registration attempt with an empty coldkey fails on-chain, so this is the single most common "register failed" cause. Give the transfer a few minutes to confirm before moving on.

> **The mnemonic never goes into the app, a screenshot, or a chat message.** Paper only. The app never asks for it, and any flow that does is not part of this workflow.

05 · STEP 3 — SETUP

Pick SN67 and run the setup from the Deploy stepper

With the machine online and the wallet funded, everything else happens on one screen: **Deployments**. The four-step stepper at the top of the page is the same wizard used for GPU subnets — for a CPU subnet it simply swaps the GPU-provider step for a compute choice, and the final step knows your burn is deliberate. The fastest entry is the **Start CPU miner** button on your machine's row in the CPU Catalog: it opens the stepper with SN67 and your laptop already preselected. Manually, it looks like this:

A

Stepper step 1 — Subnet: pick SN67 Harnyx

The row is tagged CPU-only (minimum VRAM 0). Picking it reshapes the wizard: step 2 becomes "Compute — cloud, or your laptop" and the requirement strip reads "CPU-only subnet — no GPU needed".

B

Stepper step 2 — Compute: "Local machine" → your laptop

Two target cards appear: CPU provider (rented Hetzner/DO boxes — skip it) and Local machine. Pick **Local machine**, then click your laptop from the list of online hosts. Cost shows **$0 — your hardware, your electricity**. Confirm the wallet and hotkey fields: they default to your saved wallet profile or `infranex` / `sn67miner` — set them to `harnyx` / `sn67miner` to match step 2 of this guide.

C

Stepper step 3 — "Start setup on laptop"

One click queues the whole SN67 setup as a single agent command: clone the Harnyx repo into `~/infranex/sn67`, install `uv`, fetch Python 3.11 (the repo pins `>=3.11,<3.12`) and sync the workspace including `harnyx-miner-sdk`. Live output streams into the pane; when the agent reports done, step 4 opens by itself.

```
# what "Start setup on laptop" runs on your machine (one command):
mkdir -p ~/infranex/sn67 && cd ~/infranex/sn67 \
  && (git clone https://github.com/harnyx/harnyx.git . || git pull) \
  && curl -LsSf https://astral.sh/uv/install.sh | sh \
  && uv python install 3.11 && uv sync --python 3.11 \
  && pip install -e harnyx-miner-sdk && echo SETUP-OK
```

If the setup fails (network hiccup, git conflict), the pane shows the failing lines and a **re-queue** button — fix the obvious cause if there is one and queue it again; nothing has been spent. The command feed stays available afterwards under **DevOps → Local machines**, which is also where you can run quick manual commands on the machine at any time.

06 · STEP 4 — PRE-FLIGHT

Pass the Validator Lab gate — free, before any burn

SN67's validators score agents on a visible shape, and you can read that shape **before** spending anything. Open **Validator Lab** and sync netuid 67 — the profile arrives in about fifteen seconds from the subnet's repo and the chain. The last mined shape carried **Price ≈ 56%** of reward weight with **Quality ≈ 19%**, a **~30-second** response deadline, a **~$0.50 per 1M-token** price reference and a **~25 tokens-per-second** throughput reference. Write those three numbers down: fast, cheap, good-enough with citations — that is

what wins on this subnet. Shapes change; re-sync before you trust the numbers.

> IN APP **Validator Lab** → Sync netuid 67 → read the champion & scoring shape → simulate your agent's numbers

Then use the lab as a sparring partner. Drag the sliders to your agent's real numbers — planned response time, price per million tokens, expected quality, throughput — and read the verdict: weak, below-median, or **competitive**, with the percentile position. The proof from the last pre-flight: a slow-but-deep agent (240 s, $0.60, 72% quality) scored 41.9 — weak; a tuned fast-and-cheap one scored 62.1 — competitive, roughly the 90th percentile. Iterate in minutes, for free, until the composite clears the bar. Only your CPU and your time are at stake here.

> **The gate rule that protects your TAO: no green verdict — no burn.** The register button in the next step exists precisely for the moment this gate is green, not before. If your agent is not ready yet, stop here and come back — the seat price may wiggle, but it has stayed in the cents-of-a-TAO range, and patience is cheaper than a wasted burn.

07 · STEP 5 — THE BURN

Register on-chain — the one irreversible click

Back on **Deployments**, the stepper's step 4 is the go-live gate. With setup done, it shows the **burn card**: a deliberate, clearly-labeled action that queues the registration on your laptop — nothing is sent on-chain by the app itself until you press it. Before pressing, do the three-second sanity check: the Validator Lab verdict is competitive, the burn shown in the subnet strip is still around 0.0433 τ (it floats with seat demand), and the coldkey you funded in step 2 is the wallet named on the card. Then press **Send register command to laptop**.

```
# what the button queues on your laptop (coldkey signs, hotkey claims the seat):
btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey sn67miner
```

The command consumes **~0.0433 τ (≈ $11.62)** from the coldkey and takes a seat on SN67 for your hotkey. Verify it worked: the hotkey should appear in SN67's metagraph (`btcli subnets metagraph --netuid 67`, or the Subnets view in the app), and the seat starts with an **immunity window** before it can be de-

registered by weight competition — use that window to get your agent submitted and scoring, not to celebrate. From this point your laptop is the miner's home: keep it awake, keep the agent alive, and keep the terminal (or `tmux` session) running.

> **This is the point of no return.** The burn is not refundable and the seat is not transferable. Everything before this step was free by design; everything after it earns or does not earn based on your agent's quality. If any pre-flight check is uncertain, walk back to step 4 — the burn has waited this long, it can wait another hour.

08 · STEP 6 — GO LIVE

Submit the agent and start serving

Registration claims the seat; the **agent script** is what earns on it. SN67's miner is a Python agent built inside the `harnyx-miner-sdk` harness that step 3 installed. Package your agent per the repo's own submission flow — the SDK's job, not the app's (the app deliberately has no upload path) — and submit it with your Harnyx wallet keys: the coldkey signs, the hotkey serves. Validators pick the agent up, execute it in their sandboxes against live queries, and score it every epoch.

- **Keep responses under the ~30 s deadline** — slow answers lose on Market Clearing no matter how good they are.
- **Price at or below the ~$0.50/1M reference** and hold quality ⩾ 80% with citations — price carries the majority of the reward weight.
- **Target ⩾ 25 tps throughput** — the validator rewards speed per token, not just total time.
- **Watch the first epochs** on `api.harnyx.ai/mcp` and `dashboard.harnyx.ai/benchmark` before calling it done — your agent's rank there is the ground truth the chain rewards against.

If the first epochs score poorly, do not panic and do not re-burn anywhere else: tune the agent against the Validator Lab shape (latency down, price down, quality held) and resubmit through the SDK. Iteration after registration is free — it is only the seat that cost TAO, and the seat is yours until weights push it out, which the immunity window gives you time to prevent.

09 · STEP 7 — STEADY STATE

Monitor & earn — the weekly ritual

A miner is a small business; give it a ten-minute weekly review. Emissions on SN67 move — recently ~9.8 τ/day spread across a couple hundred rewarded seats with a wide spread where the top 10% of miners hold roughly a quarter of the incentive — so mid-pack miners keep their seats, but the deal can still die slowly. The weekly check exists to catch that before it eats your ROI, and every surface you need already lives in the app you are running.

Check	Where	Healthy signal
Deal economics	Opportunities strip and the SN67 row (alpha price, emission/day, seat churn)	Alpha price stable or rising; your seat still rewarded
Agent rank	api.harnyx.ai/mcp and dashboard.harnyx.ai/benchmark	Composite at or above the competitive band (~p90 ≈ 62 on the last read)
Real payouts	taostats — your coldkey's TAO flow	Steady or growing weekly TAO arrivals
Machine health	DevOps → Local machines (load, memory, temp, agent online)	Agent online, temp < 78 °C, no failed commands
Decision rule	Opportunities + Monitoring, weekly	If net expectation stays under target for weeks — deregister and move the burn

10 · REFERENCE

Command cheat sheet & troubleshooting

Every terminal command in this workflow, in the order you will actually meet them. The app builds the enrollment and setup lines for you (with real URLs and tokens) — the ones below show the shape they take.

Action	Command (on the laptop, Ubuntu)
Windows: get Ubuntu	`wsl --install -d Ubuntu`
Enroll the agent	`curl -fsSL <app-url>/api/agent/agent.py -o /tmp/infranex-agent.py && python3 /tmp/infranex-agent.py enroll --server <app-url> --token <TOKEN>`
Run the agent	`nohup python3 /tmp/infranex-agent.py run &` (or inside tmux)
Create coldkey	`btcli wallet new-coldkey --wallet.name harnyx --wallet.path ~/.bittensor/wallets`
Create hotkey	`btcli wallet new-hotkey --wallet.name harnyx --wallet.hotkey sn67miner`
Register — BURN	`btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey sn67miner`
Verify the seat	`btcli subnets metagraph --netuid 67`

Symptom	Cause → fix
Machine stuck "awaiting enroll"	The token is one-time and expires in 30 minutes → press **Connect my laptop** again and run the fresh command.
Machine shows offline	Agent process died or laptop slept → restart python3 /tmp/infranex-agent.py run inside tmux; disable automatic sleep.
Setup failed in stepper step 3	Network or git hiccup → read the output pane, press the re-queue button; nothing was spent.
Register button disabled	Setup not reported done, or the host is offline → wait for the green setup state with the agent online.
Register failed on-chain	Coldkey not on the laptop, or insufficient TAO → repeat step 2 of this guide; check balance with btcli wallet overview.
Not earning after go-live	Agent scoring below the band → back to Validator Lab: cut latency, cut price, hold quality, resubmit via the SDK.

> **The workflow in one breath:** connect the laptop → create & fund the wallet on it → pick SN67 in the stepper and run setup → pass the free Validator Lab gate → send the one burn → submit the agent → review weekly. Free until the gate is green; one burn when it is.

Free until the gate is green. One burn when it is.

That is the whole discipline. Your laptop is the mine, the Validator Lab is the safety inspector, and the burn is the only money that ever moves — deliberate, gated, and reversible by nothing. Keep the agent alive, keep the numbers honest, and review the deal every week.

Numbers in this guide were snapshots from September 2026 (block 9,101,516 · TAO $268.33 · SN67 burn 0.0433 τ). Emissions, seat prices and validator shapes move — re-read the live Opportunities row before committing budget, and treat this guide as the method, not today's price.